Project Report

Analysis of WPA2 4-Way Handshake Vulnerability Using Python-Generated Simulated Handshake Data

Date: December 2025

1. Introduction

The WPA2 4-Way Handshake is a crucial component of Wi-Fi security, responsible for establishing cryptographic keys between client and access point. While WPA2 remains widely deployed, weaknesses such as weak passphrases or predictable key-generation patterns can expose networks to offline attacks.

In this project, instead of capturing real network traffic, we **designed Python scripts to generate realistic, structurally accurate *simulated* WPA2 4-Way Handshake packets**. This approach ensured complete ethical safety, no radio communication, and no use of unauthorized networks or devices.

The simulated handshake dataset was then analyzed using Python (Scapy-style structures) to study:

- How handshake messages are structured
- What fields are critical for authentication
- What potential weaknesses exist
- How attackers exploit poorly configured WPA2 networks

This research is **fully offline, safe, ethical, and for educational purposes only**.

---

2. Problem Statement

Majority of WPA2 weaknesses arise not from protocol failure but from **implementation and configuration mistakes**, such as:

- Weak PSKs
- Reused passphrases
- Predictable SSIDs used in PMK derivation
- Misconfigured access points

However, capturing real handshakes requires specialized hardware and can pose ethical concerns if mishandled.

Therefore, the project aims to:

1. Demonstrate the structure and flow of the WPA2 4-Way Handshake using **synthetically generated packets**.
2. Analyze fields involved in key confirmation and derivation.
3. Highlight conditions that lead to practical vulnerabilities.
4. Provide mitigation strategies for stronger wireless security.

This method avoids real-world risks while still providing deep technical insight.

---

3. Scope Change Explanation (Important)

Original Proposal:
Capture WPA2 handshakes from real AP and client devices in a controlled lab.

Final Implementation:
Due to constraints and ethical considerations:

✓ No real hardware or network capture was performed
✓ Python scripts were used to generate artificial handshake packets
✓ Analysis was 100% offline and simulated

These simulated packets accurately represented the handshake structure (ANonce, SNonce, MIC, Key Information field, etc.) allowing safe analysis without risk.

---

4. Features Implemented

4.1 Simulated Handshake Dataset

- Python scripts generated handshake messages 1–4.
- Variable test conditions:
    - Strong vs weak passphrases
    - Randomized nonce values
    - Deliberate weak/non-random nonces (to simulate vulnerabilities)
    - Changeable replay counters

4.2 Python-Based Packet Analysis

Scripts performed:

- Parsing of simulated packets
- Extraction of:
    - ANonce
    - SNonce
    - MIC
    - Replay Counter
    - Key Information flags
- PMK/ PTK derivation attempts using provided passphrase (for demonstration)
- Detection of weak parameter behavior (e.g., predictable nonces)

4.3 Visualization s Reporting

Included:

- Flow diagram of the handshake
- Annotated packet fields
- Explanation of cryptographic dependencies
- Summary tables of strong vs weak test scenarios

---

5. Methodology

5.1 Data Simulation

Using Python:

- Random byte generators created nonces.
- HMAC-SHA1 functions simulated MIC generation.
- Data structures mimicked Scapy's 802.11 EAPOL format.
- Multiple handshake sets were produced.

5.2 Analysis Process

1. Load artificial handshake objects.
2. Extract relevant fields.
3. Attempt PSK-based PMK derivation.
4. Compare results to detect:
    - Weak passphrase susceptibility
    - Incorrect nonce generation
    - Improper key descriptor settings

5.3 Ethical Safety

- No radio capture
- No real SSIDs, MAC addresses, or passphrases
- Entire project executed offline
- Fully compliant with cybersecurity ethics and university policy

---

6. Results s Findings

6.1 Weak PSK Simulation

Using a weak passphrase such as “12345678”:

- PMK derivation required minimal time
- MIC matching succeeded quickly
- Demonstrated how dictionary attacks succeed offline

6.2 Strong PSK Scenario

Using 16–20 character random passphrases:

- PMK brute force became computationally infeasible
- Demonstrated security improvement with strong passwords

6.3 Nonce Vulnerabilities

Simulated “bad” implementations included:

- Repeated nonces
- Predictable incrementing nonce patterns

This showed how nonce predictability can lead to key-reconstruction attacks (as in some past WPA2 implementation bugs).

6.4 Replay Counter Issues

Improper replay counter handling in simulation highlighted:

- Why APs must strictly validate replay counters
- How attackers can exploit replay tolerance

---

7. Discussion

Even with fully simulated data, the analysis clearly shows:

- WPA2 is structurally strong when configured correctly
- Weak passphrases remain the biggest practical risk
- Random nonce generation is critical
- WPA3 addresses many of the weaknesses through SAE

The simulation method successfully reproduced real-world weaknesses without requiring real traffic capture.

---

8. Recommendations

For Network Administrators

- Use 16+ character random passphrases
- Disable WPS
- Update router firmware regularly
- Prefer WPA3 (SAE) whenever supported

For Students / Researchers

- Perform all Wi-Fi handshake experiments offline or in simulation
- Never capture packets from unauthorized networks
- Use Python/Scapy simulations for safe learning

G. Conclusion

This project successfully analyzed the WPA2 4-Way Handshake and demonstrated practical weaknesses using **Python-generated simulated handshake packets**. The approach ensured full ethical compliance while still providing meaningful cybersecurity insights.

Key contributions of the project include:

- A safe and reproducible method to study WPA2
- Understanding of cryptographic fields within the handshake
- Demonstration of how weak configurations lead to vulnerabilities
- Clear mitigation strategies to strengthen wireless networks

The simulation-based methodology is effective, safe, and highly suitable for academic environments where real packet capture may not be feasible or ethical.

10. Deliverables

- Python scripts for handshake generation
- Python scripts for handshake parsing and analysis
- Simulated PCAP-like output files (JSON/structs)
- Final report (this document)
- Presentation slides